

Hybrid N-gram and Neural Language Model for Geonmic Sequences

Rodrigo Orozco

Electrical and Computer Engineering,
Texas A&M University
rodrigoorozco03@tamu.edu

Jesse Zheng

Computer Science and Engineering
Texas A&M University
yojoe56@tamu.edu

Abstract

Recent work has shown that n-gram language models can compliment neural language models in reducing perplexity. Such an n-gram LM is defined as an infini-gram, a generalization of the n-gram where conceptually backoff starts from $n = \infty$. When interpolating with decoder-only neural LMs, perplexity reduces compared to decoder alone. We hypothesized that such a combination could be applied in the area of genomic sequences. To achived this, we propose a bidirectional infini-gram LM that approximates masked language modeling, a popular pretraining objective among foundational models in biology. Thus we define a hybrid model as the combination of a bidirectional infini-gram with the Nucleotide Transformer, an encoder-only neural LM, via linear interpolation. We find that the bidirectional infini-gram fails to generalize robustly, and the hybrid model's low perplexity is an artifact of evaluating on a small, unrepresentative subset of the data

1 Introduction

The adoption of LM practices to the area of biology is natural as the task of predicting word tokens becomes that of predicting nucleotide bases or amino acids. Moreover, genomics is inherently aligned to the language modeling paradigm. The genomic "language"'s alphabet is the nucleotide bases, its vocabulary k-mers, its syntax and grammar are the rules that govern sequence organization, and its semantics the biological function of a sequence. The Nucleotide Transformer is a collection of encoder-only transformers varying in size and pertaining data following the BERT methodology. Similar to NLP foundational models, the NT offers general-purpose representations for genomics. These embeddings can improve performances across many downstream genomics tasks. Such a model is beneficial due to the lack of labeled data in biology and poor model transferability.

Despite the domainance of neural LMs in the field of NLP, Liu et al. (Liu et al., 2024) showed that the linear interpolation of decoder models with a generalization of n-gram LMs, the infini-gram, can reduce perplexity compared to the neural LM alone. This suggests that n-gram LMs can capture information that neural LMs miss. The infini-gram is defined as an n-gram LM where backoff begins from $n = \infty$. Backoff is applied only when the full context is unseen, thus the model uses the largest observed matching context for each prediction. Moreover, the infini-gram is powered by a performant engine for n-gram/ ∞ -gram queries. Its performance originates from the use of a suffix array index that allows counting of any substring in $O(L + \log(N))$, where L is the length of query string tokens, and N is the total number of tokens in the training corpus. Since, substring occurrences appear in one consecutive block of the suffix array, to count a substring, a binary search is performed to find the start and end of the block, and the difference of these two indices is the count.

Motivated by these findings, we propose a bidirectional infini-gram LM that approximates masked language modeling, the pretraining objective of the Nucleotide Transformer in order to combine both models via linear interpolation.

2 Approximating MLM - Bidirectional Infini-gram

Masked language modeling is a pretraining objective where a percentage of tokens in the input sequence are masked and the model is tasked with predicting these masked tokens. BERT utilizes bidirectional attention allowing tokens to attend to each other. Thus, infomration can flow in both directions and the resulting embeddings are contexttualized to both left and right contexts. N-gram LM is not well defined for MLM since the probaility distribution is over left-to-right contexts. As a result, we

propose a bidirectional infini-gram LM that approximates MLM by utilizing two infini-gram indicies.

Given a sequence with masked tokens, for a single masked token we define its left and right contexts as the tokens to the left and right of the masked token up until another mask is observed. Formally, let $\mathbf{x} = (x_1, x_2, \dots, x_L)$ be the unmasked sequence and $\mathbf{m}$ be the sequence where masked positions are denoted by $m_i = [\text{MASK}]$. For a masked position p where $1 \leq p \leq L$, we define i^* as the position of the nearest mask to the left of p (or 0 if none), and j^* as the position of the nearest mask to the right (or $L + 1$ if none):

$$\begin{aligned} i^* &= \max(\{j < p : m_j = [\text{MASK}]\} \cup \{0\}) \quad (1) \\ j^* &= \min(\{k > p : m_k = [\text{MASK}]\} \cup \{L + 1\}) \quad (2) \end{aligned}$$

The left and right contexts are then:

$$C_L(p) = (x_{i^*+1}, \dots, x_{p-1}) \quad (3)$$

$$C_R(p) = (x_{p+1}, \dots, x_{j^*-1}) \quad (4)$$

Using the two infini-gram indices, the next token distribution is obtained from each index using the left and right contexts respectively. Let $P_{\text{fwd}}(\cdot \mid C_L(p))$ denote the next-token distribution from the forward infini-gram conditioned on the left context, and $P_{\text{bwd}}(\cdot \mid C_R(p)^{\text{rev}})$ denote the distribution from the backward infini-gram conditioned on the reversed right context. These distributions are computed using the infini-gram engine and normalized to ensure:

$$\sum_{v \in \mathcal{V}} P_{\text{fwd}}(v \mid C_L(p)) = 1 \quad (5)$$

$$\sum_{v \in \mathcal{V}} P_{\text{bwd}}(v \mid C_R(p)^{\text{rev}}) = 1 \quad (6)$$

Each index returns next token candidates, tokenIDs with non-zero probabilities, these candidates are then unioned.

$$\begin{aligned} \mathcal{C}(p) &= \{v : P_{\text{fwd}}(v \mid C_L(p)) > 0\} \cup \\ &\quad \{v : P_{\text{bwd}}(v \mid C_R(p)^{\text{rev}}) > 0\} \quad (7) \end{aligned}$$

To get a score, each of the candidates' log-probabilities from each distribution are combined and weighted by how many tokens were visible on each side. Let $|C_L(p)|$ and $|C_R(p)|$ denote the lengths of the left and right contexts respectively.

We define the interpolation weights as:

$$\alpha = \begin{cases} \frac{|C_L(p)|}{|C_L(p)| + |C_R(p)|} & \text{if } |C_L(p)| + |C_R(p)| > 0 \\ 0.5 & \text{otherwise} \end{cases} \quad (8)$$

and $\beta = 1 - \alpha$. The score for each candidate token $v \in \mathcal{C}(p)$ is computed as:

$$\begin{aligned} s(v) &= \alpha \log P_{\text{fwd}}(v \mid C_L(p)) + \\ &\quad \beta \log P_{\text{bwd}}(v \mid C_R(p)^{\text{rev}}) \quad (9) \end{aligned}$$

To avoid undefined logarithms, we replace zero probabilities with 10^{-12} .

Using the scores we create a probability distribution via softmax. The final bidirectional probability distribution is:

$$P(v \mid C_L, C_R) = \frac{\exp(s(v) - s_{\max})}{\sum_{v' \in \mathcal{C}(p)} \exp(s(v') - s_{\max})} \quad (10)$$

where $s_{\max} = \max_{v \in \mathcal{C}(p)} s(v)$ is used for numerical stability.

The masked token is predicted as the token with the highest probability from the resulting distribution:

$$\hat{x}_p = \arg \max_{v \in \mathcal{C}(p)} P(v \mid C_L, C_R) \quad (11)$$

3 Hybrid Model for MLM

Similar to (Liu et al., 2024), the hybrid model is defined as the linear interpolation of the bidirectional infini-gram and the Nucleotide Transformer. We introduce two hyperparameters for sparse and dense contexts, and a threshold to select between them based on the maximum context seen by the infini-gram.

$$\lambda_{\text{NT}} = \begin{cases} \lambda_{\text{sparse}} & \text{if } n_{\text{ctx}} < \tau \\ \lambda_{\text{dense}} & \text{otherwise} \end{cases} \quad (12)$$

where τ is a threshold parameter, λ_{sparse} assigns higher weight to the neural model for sparse contexts, and λ_{dense} places more trust in the infini-gram for dense contexts.

The hybrid distribution is computed via linear interpolation for the union of tokens with non-zero probability from either model:

$$\begin{aligned} \tilde{P}_{\text{hybrid}}(v) &= \lambda_{\text{NT}} P_{\text{NT}}(v \mid p) + \\ &\quad (1 - \lambda_{\text{NT}}) P_{\text{IG}}(v \mid C_L, C_R) \quad (13) \end{aligned}$$

We then renormalize the distribution:

$$P_{\text{hybrid}}(v) = \frac{\tilde{P}_{\text{hybrid}}(v)}{\sum_{v' \in \mathcal{T}} \tilde{P}_{\text{hybrid}}(v')} \quad (14)$$

In cases where the infini-gram has no support the model falls back to the pure Nucleotide Transformer distribution: $P_{\text{hybrid}}(v) = P_{\text{NT}}(v | p)$.

4 Fine-Tuning Nucleotide Transformer with IA3

Following the IA3 formulation introduced in the Nucleotide Transformer paper (Section A.1.4; Dalla-Torre et al., 2023), we treat the 500M-parameter encoder as a frozen backbone and introduce a small set of task-specific rescaling parameters. In a standard transformer layer, self-attention is computed as

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (15)$$

$$\text{Attn}(X) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad (16)$$

and the feed-forward network (FFN) takes the form

$$h = \sigma(W_1X + b_1), \quad (17)$$

$$\text{FFN}(X) = W_2h + b_2, \quad (18)$$

where $X \in \mathbb{R}^{T \times d}$ denotes the hidden states, d_k is the key dimension, and $\sigma(\cdot)$ is a nonlinearity.

Full fine-tuning updates all weights W_Q, W_K, W_V, W_1, W_2 in every layer. In contrast, IA3 introduces *diagonal* rescaling vectors and keeps these large weight matrices fixed. For each transformer layer ℓ , IA3 defines three learnable vectors

$$\ell_k^{(\ell)} \in \mathbb{R}^{d_k}, \quad \ell_v^{(\ell)} \in \mathbb{R}^{d_v}, \quad \ell_{\text{ff}}^{(\ell)} \in \mathbb{R}^{d_{\text{ff}}}, \quad (19)$$

which modulate the internal activations via element-wise multiplication:

$$K'^{(\ell)} = K^{(\ell)} \odot \ell_k^{(\ell)}, \quad (20)$$

$$V'^{(\ell)} = V^{(\ell)} \odot \ell_v^{(\ell)}, \quad (21)$$

$$h'^{(\ell)} = h^{(\ell)} \odot \ell_{\text{ff}}^{(\ell)}, \quad (22)$$

where $\odot$ denotes broadcasting and element-wise multiplication. Intuitively, large entries in these vectors *amplify* specific channels, while small entries *inhibit* them. The pretrained transformer thus acts as a rich feature extractor, and IA3 learns a task-specific gating pattern over these features rather than relearning the features themselves.

In our implementation, each encoder block is associated with an IA3Layer module that stores

the three vectors $\ell_k^{(\ell)}, \ell_v^{(\ell)}, \ell_{\text{ff}}^{(\ell)}$, all initialized to one so that the initial model behaves like the frozen Nucleotide Transformer. To keep the implementation simple and lightweight, we aggregate these per-layer rescaling parameters into a single global scaling factor applied to the final hidden representation. Concretely, letting $H \in \mathbb{R}^{T \times d}$ denote the final encoder hidden states and $\alpha \in \mathbb{R}$ denote the learned global rescaling factor derived from the IA3 parameters, we use

$$\tilde{H} = \alpha H, \quad (23)$$

and extract the [CLS] representation from $\tilde{H}$ for classification. Only the IA3 parameters and the small classification head are updated during training, yielding a parameter-efficient fine-tuning scheme in which roughly 0.1% of the total model parameters are task-specific. This allows us to adapt a large genomic foundation model to promoter recognition while keeping the memory and optimization footprint manageable on the available Grace GPUs.

5 Hybrid Model for Promoter Classification

In the hybrid setting, the fine-tuned Nucleotide Transformer serves as the neural backbone. Given a 300 bp input sequence, the model produces a dense [CLS] embedding, while a separate Infini-gram pipeline computes complementary n -gram statistics on the same sequence. We then concatenate the Nucleotide Transformer embedding with these Infini-gram features and pass the combined representation through an MLP classifier, allowing the hybrid model to exploit both contextual embeddings and explicit n -gram information for promoter prediction.

Following Liu et al. (Liu et al., 2024), we can also view this architecture through the lens of interpolation between neural and ∞ -gram estimates. Let $p_{\text{NT}}(t)$ denote the probability assigned to token t by the Nucleotide Transformer head, and let $p_{\infty}(t)$ denote the corresponding ∞ -gram probability. We define a context sparsity score c (for instance, based on the maximum forward/backward ∞ -gram agreement for the masked position) and compare it to a sparsity threshold τ . If $c < \tau$, the context is treated as *sparse*, otherwise as *non-*

sparse. The hybrid distribution is then

$$p_{\text{hybrid}}(t) = \begin{cases} \lambda_s p_{\text{NT}}(t) + (1 - \lambda_s) p_{\infty}(t), & c < \tau \\ \text{(sparse)} \\ \lambda_d p_{\text{NT}}(t) + (1 - \lambda_d) p_{\infty}(t), & c \geq \tau \\ \text{(non-sparse)}, \end{cases} \quad (24)$$

where $\lambda_s, \lambda_d \in [0, 1]$ are interpolation hyperparameters for sparse and dense contexts, respectively. In practice, λ_s, λ_d , and the sparsity threshold τ are tuned on the promoter validation set. This formulation mirrors the sparse/non-sparse interpolation strategy proposed for text LMs in Infini-gram (Liu et al., 2024), but applied here to genomic sequences and our bidirectional Infini-gram LM.

6 Methodology

To determine whether the infini-gram captures information that the Nucleotide Transformer can benefit from, we evaluate the performances of the 500M-human-reference Nucleotide Transformer model, the bidirectional infini-gram, and the hybrid model on the reconstruction of masked tokens in genomic sequences. Additionally, to determine whether such a combination is useful for downstream tasks we evaluate the performances of the 500M-human-reference and the hybrid model on promoter vs non-promoter classification.

6.1 MLM Reconstruction

From the models in the Nucleotide Transformer collection, the 500M-human-reference model was selected since it was the smallest and was trained on the human reference genome (GRCh38/hg38 dataset) allowing us to build infini-gram indices solely based on one dataset. We found a corresponding script in hugging face script () used to build the train, validate, and test split from GRCh38/hg38. We integrated this script into the repo.

The 500M-human-ref model was evaluated on the full test set and later post-submission on 300 randomly selected sequences from the test set.

To build the infini-gram indices, we made minor modifications to the source code provided by (Liu et al., 2024) to support the Nucleotide Transformer tokenizer. Following the infini-gram package documentation, the training data was converted to jsonl format to construct the forward index. To obtain the backward index, we reversed the jsonl sequences and applied the same indexing procedure. Using

these two indices, we instantiated the bidirectional infini-gram model.

After implementing the bidirectional infini-gram class, we observed that evaluating the full validation set during hyperparameter tuning was computationally expensive. Consequently, instead of tuning on the full validation set, we limited our experiments to smaller random samples. For each direction, we introduced a tunable hyperparameter, max support, which controls the maximum number of token IDs returned by the infini-gram engine with non-zero probability in the next-token distribution. We performed a grid search over these hyperparameters using 300 randomly selected sequences from the validation set. After selecting the optimal settings, we built two additional indices that used the combined training and validation data, and evaluated performance on 300 randomly selected sequences from the test set. Although this reduced the runtime, it was still insufficient to fully explore the hybrid model within the available time.

Following submission, we were able to parallelize the bidirectional infini-gram which allowed us to evaluate performance on the full test set.

After constructing the bidirectional infini-gram, we tuned the hybrid model hyperparameters using 20 randomly selected validation sequences and evaluated the resulting system on 300 test sequences. As a result of the computational constraints, hyperparameter tuning for the hybrid model was performed on only 20 validation sequences, and its final evaluation was conducted on 300 randomly selected test sequences. These hybrid model experiments did not complete until after the code and presentation deadline. Moreover, after the submission, we attempted to parallelize the bidirectional infini-gram component in order to evaluate the full test set. However, this proved challenging, as the transformer model simultaneously required access to the GPU, making efficient parallelization difficult.

To evaluate the models' performance on reconstructing masked tokens in a genomic sequence we followed (Dalla-Torre et al., 2023) performance analysis section (A.5.1). Within a nucleotide sequence of interest, 15% of tokens were randomly masked. The masked sequence was then given to each of the models and the probabilities over tokens at each masked position were retrieved. We evaluated masked token reconstruction using loss, accuracy, and an alternative definition of perplexity. We defined accuracy as the fraction of correctly

predicted masked tokens in a sequence:

$$\text{acc}(\theta, \mathbf{s}) = \frac{1}{|\mathcal{P}_{\text{masked}}|} \sum_{i \in \mathcal{P}_{\text{masked}}} \mathbf{1}(\hat{v}_i = \mathbf{s}(i)) \quad (25)$$

where $\hat{v}_i = \arg \max_{v \in \mathcal{V}} \log p(\theta, i, v)$ is the predicted token at position i . Loss as the negative log likelihood of predicting the correct token:

$$l(\theta, \mathbf{s}) = - \sum_{i \in \mathcal{P}_{\text{masked}}} \sum_{\text{tok} \in \mathcal{V}} \log p(\theta, i, \text{tok}) \cdot \mathbf{1}(\text{tok} = \mathbf{s}(i)) \quad (26)$$

and perplexity as exponential of the loss function computed over the masked positions:

$$\text{perplexity}(\theta, \mathbf{s}) = 2^{l(\theta, \mathbf{s})} \quad (27)$$

Where θ represents the parameters of a trained LM, and $\mathbf{s}$ a nucleotide sequence of interest.

Note: we found a bug in the code submitted that computes the log in base e rather than base 2. The values presented in the results section correct this mistake by computing perplexity with base e.

6.2 Promoter Classification

For the promoter-vs.-non-promoter classification task, we adapt the Human Ref 500M Nucleotide Transformer (Dalla-Torre et al., 2023) to a binary classifier using the IA3 scheme described in Section 4. We use the DeePromoter / Nucleotide Transformer benchmark dataset, consisting of approximately 59,000 sequences, with positive examples defined as windows from -249 to $+50$ bp around annotated transcription start sites and negative examples constructed by shuffling promoter segments.

On the Texas A&M Grace high-performance computing (HPC) cluster, we requested a GPU node with A100 accelerators via Slurm and configured the software environment specifically for transformer training. In the job script, we selected a GPU partition and requested two A100 GPUs (e.g., `#SBATCH -gres=gpu:a100:2`) along with sufficient CPU cores and memory. Inside the job, we loaded the relevant CUDA/PyTorch and Python modules, then activated a dedicated virtual or conda environment containing torch, transformers, datasets, and peft for IA3 support. The promoter sequences (300 bp each) were preprocessed into FASTA/CSV format and staged on node-local storage to reduce I/O overhead during training.

To leverage both A100 GPUs, we used PyTorch DistributedDataParallel (DDP). At job start, the training script initialized a process group using NCCL as the backend and spawned one training process per GPU. Each process constructed the same Nucleotide Transformer + IA3 model, wrapped it in DistributedDataParallel, and iterated over a DataLoader with a DistributedSampler so that each GPU saw a unique shard of the mini-batches. The model consumed tokenized 300 bp sequences and produced a [CLS] embedding, which was passed into a small feed-forward classifier predicting promoter vs. non-promoter labels. For optimization, we used Adam with a learning rate of 3×10^{-3} , a global batch size of 8, and trained for 10,000 steps on stratified training folds, matching the configuration used in our slides.

7 Results

7.1 Intrinsic Evaluation: Masked Token Reconstruction

The bidirectional infini-gram hyperparameters were tuned using grid search over the max support values for both forward and backward directions. As explained in the methodology section, computational constraints limited the exploration to 300 randomly selected validation sequences.

Max Support		Metrics			
Fwd	Bwd	Loss	Acc.	Med. Acc.	Perp.
8	8	7.908	0.540	0.540	2702.5
8	16	7.890	0.540	0.540	2666.5
16	8	7.890	0.540	0.540	2666.5
16	16	7.897	0.540	0.540	2685.3

Table 1: Grid search results for bidirectional infini-gram hyperparameters

Based on the results from hypertuning, we inferred that the hyperparameter values we selected had little to no effect on the metrics and decided to use max support of 8 on the test set as it the cheapest computationally.

The hybrid model’s hyperparameters were tuned on 20 randomly selected validation sequences via grid search. The results indicate that only λ_{dense} significantly affects performance, with $\lambda_{\text{dense}} = 0.5$ yielding better results across all metrics. The hyperparameters λ_{sparse} and τ showed no observable effect on the outcomes. Moreover, these particular results may not hold since we accidentally committed data leakage. Since the Nucleotide Transformer

Hyperparameters			Metrics			
λ_s	λ_d	τ	Loss	Acc.	Med. Acc.	Perp.
0.7	0.2	2	0.848	0.538	0.537	2.334
0.7	0.2	7	0.848	0.538	0.537	2.334
0.7	0.5	2	0.446	0.571	0.573	1.562
0.7	0.5	7	0.446	0.571	0.573	1.562
0.8	0.2	2	0.848	0.538	0.537	2.334
0.8	0.2	7	0.848	0.538	0.537	2.334
0.8	0.5	2	0.446	0.571	0.573	1.562
0.8	0.5	7	0.446	0.571	0.573	1.562

Table 2: Grid search results for hybrid model hyperparameters

has seen the validation set during training, the reported performance is likely inflated. The correct methodology would have been to use the full validation set to build the infini-gram indicies, and then split the test set further that way we ensure both models have not seen the masked sequences allowing for honest hyperparameter tuning in the hybrid model.

After determining the hyperparameters for the bidirectional infini-gram and hybrid model, we evaluated all three models on 300 randomly selected sequences from the test set. Table 3 presents these results.

Model	Loss	Acc.	Med. Acc.	Perp.
NT (500M)	4.533	0.359	0.167	93.0
Bidir Infini-gram	7.916	0.540	0.540	2736.8
Hybrid	0.444	0.570	0.573	1.558

Table 3: Test set comparison on 300 randomly selected sequences

Post submission, we evaluated the bidirectional infini-gram on the full test set and compared it with the 500M-human-ref.

Model	Loss	Acc.	Med. Acc.	Perp.
NT (500M-hf)	4.398	0.376	0.187	81.3
Bidir Infini (max=4)	7.163	0.279	0.053	1292.3
Bidir Infini (max=8)	8.972	0.176	0.060	7890.6

Table 4: Full test set results (8,469 sequences)

Notably, the bidirectional infini-gram’s performance degrades significantly on the full test set compared to the 300-sequence subset. With a maximum support of 4, accuracy drops from 54.0% to 27.9%, and with a maximum support of 8, it plummets to 17.6%. This dramatic performance difference suggests that the 300-sequence validation subset may not be representative of the full test distribution, highlighting the importance of evaluating on comprehensive test sets. Thus, the results for the hybrid model in Table 3 do not accurately depict

its performance. Unfortunately, parallelizing the bidirectional infini-gram and utilizing the GPU for the transformer proved challenging and could not be completed before the report deadline.

7.2 Extrinsic Evaluation: Promoter Classification

We next evaluated our models on the extrinsic task of binary classification of 300 bp sequences into promoter vs. non-promoter. The dataset consists of approximately 59,000 sequences drawn from the DeepPromoter / Nucleotide Transformer benchmark, with positive examples defined as windows from -249 to $+50$ bp around annotated transcription start sites and negative examples constructed by shuffling promoter segments. We split the data into an 80% train+validation portion (using k -fold cross-validation) and a held-out 20% test set, preserving class balance in each split.

Our primary baseline is the Human Ref 500M Nucleotide Transformer model (Dalla-Torre et al., 2023) fine-tuned with IA3 adapters on the Texas A&M Grace cluster (Section 6.2). After 10,000 training steps with a global batch size of 8 and the Adam optimizer with learning rate 3×10^{-3} , the IA3-augmented Nucleotide Transformer achieved an average classification accuracy of approximately 63% on the held-out test set. The original Nucleotide Transformer work does not report this exact metric on our specific promoter dataset, so a direct quantitative comparison is not possible. Nonetheless, these results demonstrate that IA3 fine-tuning can adapt a large DNA foundation model to promoter recognition with moderate performance under limited compute.

For evaluation, we consider standard classification metrics: overall accuracy, F1 score, area under the ROC curve (AUROC), and area under the precision-recall curve (AUPRC), with a particular emphasis on F1 and AUPRC due to their focus on the positive (promoter) class. In the current work, we report accuracy as our primary metric and leave a comprehensive analysis of F1, AUROC, and AUPRC across different decision thresholds to future experiments.

7.3 Hybrid NT + Infini-gram: Preliminary Findings

Our intended extrinsic evaluation for the hybrid model followed the architecture in Section 5: given a 300 bp input, we obtain a [CLS] embedding from the fine-tuned Nucleotide Transformer and concate-

nate it with Infini-gram-derived n -gram features, feeding the combined representation into an MLP classifier. Conceptually, this design would allow the classifier to integrate global contextual information from the transformer with highly local, count-based statistics from the ∞ -gram engine, mirroring the improvements observed in natural language models with Infini-gram interpolation.

However, in practice, the coplexity of developing an efficient hybrid model made this exploration infeasible. Moreover, preliminary runs on small subsets did not yield stable, reproducible gains over the IA3 Nucleotide Transformer baseline, and were heavily constrained by runtime and memory limits on the available Grace nodes. Given these limitations, we treat the hybrid promoter classification results as inconclusive and focus instead on documenting the engineering and modeling challenges encountered when attempting to integrate ∞ -gram statistics into large genomic LMs.

7.4 Summary of Results

Our experiments reveal significant challenges in adapting Infini-gram-style n -gram models to genomic sequence modeling, with results that expose fundamental methodological issues rather than demonstrating clear improvements over neural baselines.

On the 300-sequence test subset, the bidirectional infini-gram achieved 54.0% accuracy compared to the Nucleotide Transformer’s 35.9%, and the hybrid model reached 57.0%. However, these promising results did not generalize: when evaluated on the full test set of 8,469 sequences, the bidirectional infini-gram’s accuracy collapsed to 27.9% (max support = 4) or 17.6% (max support = 8), while the NT baseline maintained consistent performance at 37.6%. This dramatic degradation reveals that the 300-sequence subset was unrepresentative and that hyperparameters tuned on small samples severely overfit.

The hybrid model’s unusually low perplexity, coupled with only modest accuracy gains, is likely attributable to the limited and unrepresentative sample used for evaluation. Additionally, we discovered a logarithm base mismatch in our evaluation code, which we corrected in the reported values but which underscores the fragility of our implementation.

For promoter classification, IA3 fine-tuning of the 500M Nucleotide Transformer achieved approximately 63% test accuracy under modest com-

pute constraints, demonstrating the viability of parameter-efficient adaptation. However, computational limitations prevented thorough evaluation of the hybrid model on this extrinsic task, leaving the question of whether Infini-gram features improve downstream performance unresolved.

8 Conclusion

In this work, we investigated whether unbounded n -gram statistics from Infini-gram could enhance a genomic language model for masked language modeling and promoter classification. We implemented a bidirectional Infini-gram to approximate MLM by combining forward and backward suffix array indices, and constructed a hybrid system that interpolates between the Infini-gram and neural model predictions. While initial results on small test samples suggested improvements, comprehensive evaluation revealed critical flaws in our experimental design.

The bidirectional infini-gram’s severe performance degradation when scaling from 300 to 8,469 test sequences demonstrates that n -gram approaches trained on the full genome still fail to generalize robustly to held-out sequences from the same distribution. In contrast, the Nucleotide Transformer maintained stable accuracy across different test set sizes, suggesting that learned contextual representations are more reliable than count-based statistics for this domain. The hybrid model’s exceptional performance on small samples appears to be driven by the limited and unrepresentative evaluation subset rather than genuine complementarity between the two approaches.

Our results expose several fundamental challenges in applying Infini-gram to genomics. First, adapting a left-to-right n -gram model to bidirectional MLM is conceptually awkward and computationally expensive, requiring separate forward/backward indices and careful probability renormalization. Second, genomic sequences exhibit different statistical properties than natural language: while rare n -grams in text often correspond to meaningful semantic units, rare genomic subsequences may simply reflect biological noise or sequencing artifacts. Third, pretraining both the n -gram index and neural model on the same genome creates unavoidable train-validate overlap, making fair comparison difficult without access to entirely independent genomic datasets.

Despite these negative findings, our work makes

several contributions. We provide a cautionary case study on the risks of under-sampled evaluation and data leakage when tuning hyperparameters in genomic modeling. We demonstrate that IA3 adapters enable efficient fine-tuning of large genomic transformers for classification tasks with limited compute.

References

- Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza Revilla, Nicolas Lopez Carranza, Adam Henryk Grzywaczewski, Francesco Oteri, Christian Dallago, Evan Trop, Hassan Sirelkhatim, Guillaume Richard, and others. 2023. The Nucleotide Transformer: Building and Evaluating Robust Foundation Models for Human Genomics. *bioRxiv*, pages 2023–01. Cold Spring Harbor Laboratory.
- Jiacheng Liu, Sewon Min, Luke Zettlemoyer, Yejin Choi, and Hannaneh Hajishirzi. 2024. Infini-gram: Scaling Unbounded n-gram Language Models to a Trillion Tokens. *arXiv preprint arXiv:2401.17377*.